

```
import numpy as np
from sklearn.linear_model import LinearRegression, LogisticRegression
```

```
X = np.array([[1],[2],[3],[4],[5],[6],[7],[8]])
y_reg = np.array([30, 40, 45, 55, 65, 70, 80, 90])
y_cls = np.array([ 0, 0, 0, 0, 1, 1, 1, 1])
```

```
lin = LinearRegression()
lin.fit(X, y_reg)
print('Regression output:', lin.predict([[5.5]]))
```

Regression output: [67.76785714]

```
log = LogisticRegression()
log.fit(X, y_cls)
print('Classification output:', log.predict([[5.5]]))
```

Classification output: [1]

```
print('Probability output :', log.predict_proba([[5.5]]))
```

Probability output : [[0.2368932 0.7631068]]

```
import numpy as np
import matplotlib.pyplot as plt

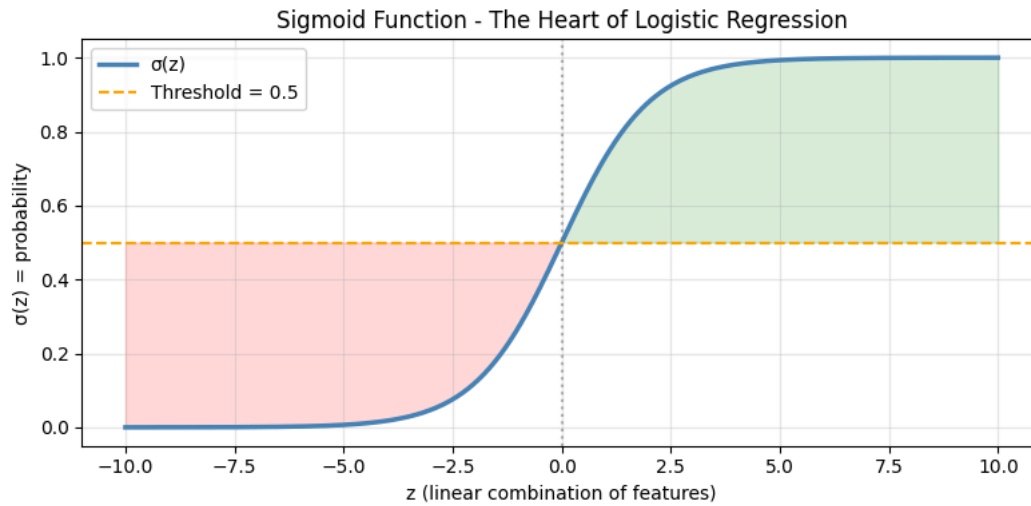
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

z = np.linspace(-10, 10, 500)
prob = sigmoid(z)
plt.figure(figsize=(8, 4))
plt.plot(z, prob, color='steelblue', linewidth=2.5, label='σ(z)')
plt.axhline(y=0.5, color='orange', linestyle='--', label='Threshold = 0.5')
plt.axvline(x=0, color='gray', linestyle=':', alpha=0.6)

plt.fill_between(z, prob, 0.5, where=(prob > 0.5),
                 alpha=0.15, color='green')
plt.fill_between(z, prob, 0.5, where=(prob < 0.5),
                 alpha=0.15, color='red')

plt.xlabel('z (linear combination of features)')
plt.ylabel('σ(z) = probability')
plt.title('Sigmoid Function - The Heart of Logistic Regression')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

print(f"sigmoid(0) = {sigmoid(0):.4f}")
print(f"sigmoid(5) = {sigmoid(5):.4f}")
print(f"sigmoid(-5) = {sigmoid(-5):.4f}")
```



```
sigmoid(0) = 0.5000
sigmoid(5) = 0.9933
sigmoid(-5) = 0.0067
```

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score, recall_score, f1_score
from sklearn.model_selection import train_test_split
```

```
X,y=make_classification(n_samples=500,n_features=5,weights=[0.7,0.3],random_state=42)
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

```
model = LogisticRegression()
model.fit(X_train, y_train)
probs=model.predict_proba(X_test)[: ,1]
```

```
print(f'{'Threshold':>10}{ 'Precision':>10}{ 'Recall':>10}{ 'F1':>10}')
print('_ '*45)
for threshold in [0.2,0.3,0.4,0.5,0.6,0.7,0.8]:
    preds=(probs>threshold).astype(int)
    p=precision_score(y_test,preds,zero_division=0)
    r=recall_score(y_test,preds)
    f1=f1_score(y_test,preds,zero_division=0)
    print(f'{threshold:>10.1f} {p:>10.3f} {r:>8.3f} {f1:>6.3f}')
```

Threshold	Precision	Recall	F1
0.2	0.696	0.941	0.800
0.3	0.762	0.941	0.842
0.4	0.882	0.882	0.882
0.5	0.882	0.882	0.882
0.6	0.903	0.824	0.862
0.7	0.903	0.824	0.862
0.8	0.958	0.676	0.793

```
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
```

```
y_true = [1,1,1,1,1,0,0,0,0,0,1,0,1,0,1]
y_pred=[1,1,1,0,0,0,0,0,1,1,1,0,0,1,1]
```

```
y_true=np.array(y_true)
y_pred=np.array(y_pred)
```

```
TP=((y_true==1) & (y_pred==1)).sum()
TN=((y_true==0) & (y_pred==0)).sum()
FP=((y_true==0) & (y_pred==1)).sum()
FN=((y_true==1) & (y_pred==0)).sum()
```

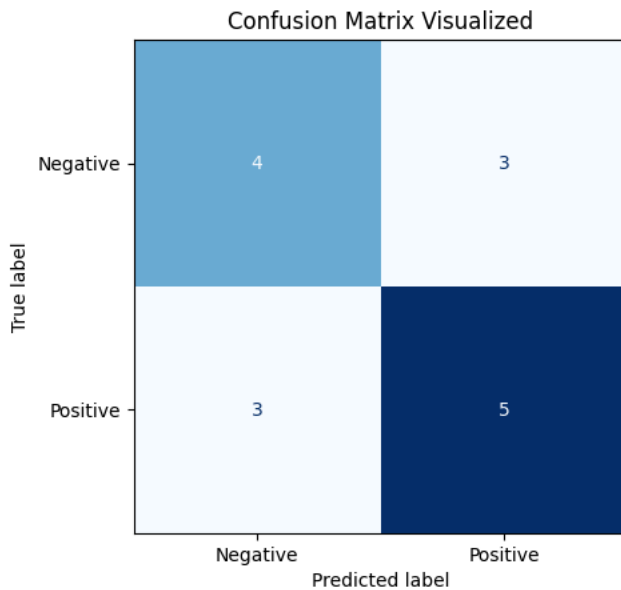
```
print(f"TP={TP}, TN={TN}, FP={FP}, FN={FN}")
print(f"Accuracy = (TP+TN)/Total = ({TP}+{TN})/{len(y_true)} = {(TP+TN)/len(y_true):.2%}")
print(f"Precision = TP/(TP+FP) = {TP}/{TP+FP} = {(TP/(TP+FP)):.2%}")
print(f"Recall = TP/(TP+FN) = {TP}/{TP+FN} = {(TP/(TP+FN)):.2%}")
```

```
TP=5, TN=4, FP=3, FN=3
Accuracy = (TP+TN)/Total = (5+4)/15 = 60.00%
Precision = TP/(TP+FP) = 5/(5+3) = 62.50%
Recall = TP/(TP+FN) = 5/(5+3) = 62.50%
```

```
cm = confusion_matrix(y_true, y_pred)
print(cm)
```

```
[[4 3]
 [3 5]]
```

```
disp = ConfusionMatrixDisplay(cm, display_labels=['Negative', 'Positive'])
disp.plot(cmap='Blues', colorbar=False)
plt.title('Confusion Matrix Visualized')
plt.show()
```



```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, f1_score, accuracy_score
from sklearn.model_selection import train_test_split
import numpy as np
```

```
X_bal, y_bal = make_classification(
    n_samples=1000,
    weights=[0.5, 0.5],
    random_state=1
)
```

```
X_imb, y_imb = make_classification(
    n_samples=1000,
    weights=[0.99, 0.1],
    random_state=1
)
```

```
def evaluate(X,y,label):
    X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
    model=LogisticRegression(max_iter=1000).fit(X_train,y_train)
    y_pred=model.predict(X_test)
    acc=accuracy_score(y_test,y_pred)
    f1=f1_score(y_test,y_pred,zero_division=0)
    print(f'\n==={label}===')
    print(f'Accuracy:{acc:.2%}<-{'MISLEADING!' if f1<0.5 else 'OK'}')
```

```
print(f'F1 score:{f1:.2%} <- Real performance')
print(classification_report(y_test,y_pred,zero_division=0))
```

```
evaluate(X_bal, y_bal, 'balanced')
evaluate(X_imb, y_imb, 'imbalanced')
```

```
===balanced===
Accuracy:80.50%<-OK
F1 score:80.00% <- Real performance
```

	precision	recall	f1-score	support
0	0.84	0.78	0.81	106
1	0.77	0.83	0.80	94
accuracy			0.81	200
macro avg	0.81	0.81	0.80	200
weighted avg	0.81	0.81	0.81	200

```

===imbalanced===
Accuracy:94.50%<-MISLEADING!
F1 score:26.67% <- Real performance
```

	precision	recall	f1-score	support
0	0.94	1.00	0.97	187
1	1.00	0.15	0.27	13
accuracy			0.94	200
macro avg	0.97	0.58	0.62	200
weighted avg	0.95	0.94	0.93	200

```
X_train,X_test,y_train,y_test = train_test_split(X_imb,y_imb,test_size=0.2,random_state=42)
model_fixed=LogisticRegression(class_weight='balanced',max_iter=1000)
model_fixed.fit(X_train,y_train)
y_pred=model_fixed.predict(X_test)
```

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_curve,roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

```
X,y = make_classification(n_samples=1000,n_features=10,random_state=42)
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

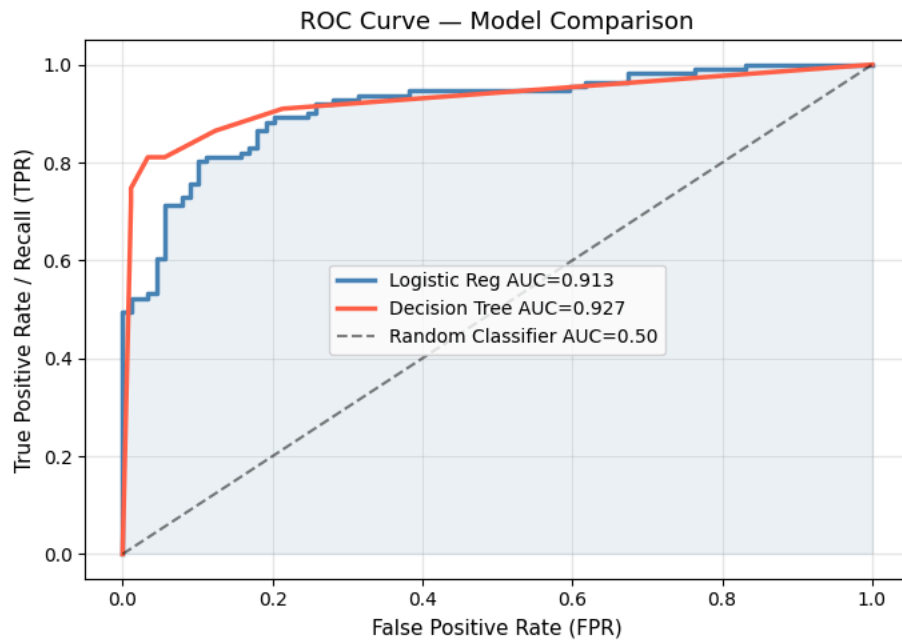
```
lr = LogisticRegression().fit(X_train,y_train)
proba_lr=lr.predict_proba(X_test)[: ,1]
```

```
dt = DecisionTreeClassifier(max_depth=4).fit(X_train, y_train)
proba_dt = dt.predict_proba(X_test)[: , 1]
```

```
fpr_lr,tpr_lr,_=roc_curve(y_test,proba_lr)
fpr_dt,tpr_dt,_=roc_curve(y_test,proba_dt)
auc_lr=roc_auc_score(y_test,proba_lr)
auc_dt=roc_auc_score(y_test,proba_dt)
```

```
plt.figure(figsize=(7, 5))
plt.plot(fpr_lr, tpr_lr, lw=2.5, color='steelblue', label=f'Logistic Reg AUC={auc_lr:.3f}')
plt.plot(fpr_dt, tpr_dt, lw=2.5, color='tomato', label=f'Decision Tree AUC={auc_dt:.3f}')
plt.plot([0,1],[0,1], 'k--', alpha=0.5, label='Random Classifier AUC=0.50')
plt.fill_between(fpr_lr, tpr_lr, alpha=0.1, color='steelblue')
plt.xlabel('False Positive Rate (FPR)', fontsize=11)
```

```
plt.ylabel('True Positive Rate / Recall (TPR)', fontsize=11)
plt.title('ROC Curve - Model Comparison', fontsize=13)
plt.legend(fontsize=10); plt.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```



Start coding or [generate](#) with AI.